PRISTINI
SCHOOL OF AI

COURSE OF

OBJECT ORIENTED PROGRAMMING - JAVA



COURSE LEADER: DR. SOUMAYA TRABELSI

LEVEL: BACHELOR-GLSI2-BI2

ACADEMIC YEAR: 2026-2027

PRISTINI

MODULE PRESENTATION

Goals :

- Know the fundamental concepts of object languages
- Study and master an object language: Java

COURSE OUTLINE

- Introduction
- Object Oriented Programming: basic concepts (syntax, types, variables, operators, etc.)
- The principles of Object Oriented Programming: Object, classes, access.
- Inheritance and Polymorphism
- Interfaces

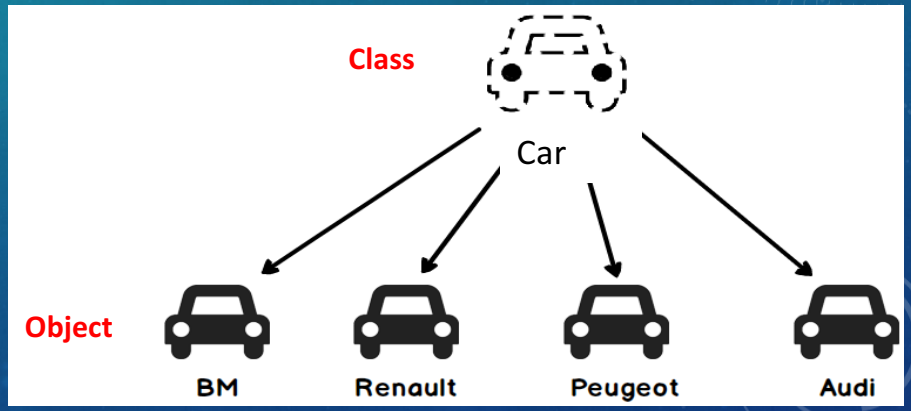
TRAINING ACHIEVEMENTS

- Use the **Java** language to implement and test algorithms to solve simple problems,
- Design and implement a **class**,
- Use object-oriented **encapsulation** mechanisms such as **interfaces** and **private members**,
- Apply decomposition techniques to break a complex program into simpler, **reusable** pieces,
- Use **inheritance** to design simple class hierarchies allowing subclasses to reuse code,

- Reason correctly about control flow in a program involving dynamic binding.

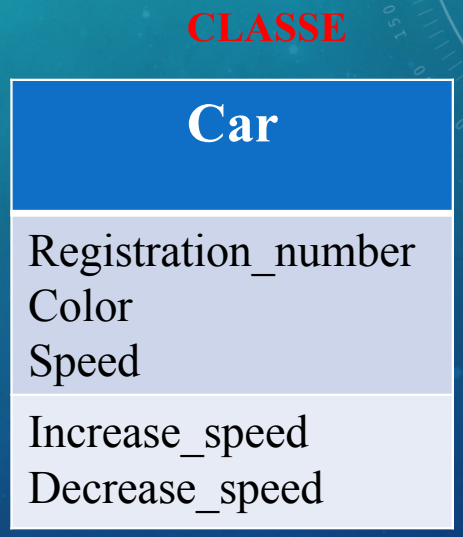
OBJECT-ORIENTED PROGRAMMING

- Object-oriented programming involves representing real-world elements through computer entities called **Objects** by adapting a high level of abstraction.
- An object is an instance of a class.



OBJECT-ORIENTED PROGRAMMING

- Each object is characterized by data (**attributes**) and processes (methods) which manipulate this data.
- An application is represented by objects in interaction.
- Data-driven programming, not process-driven.

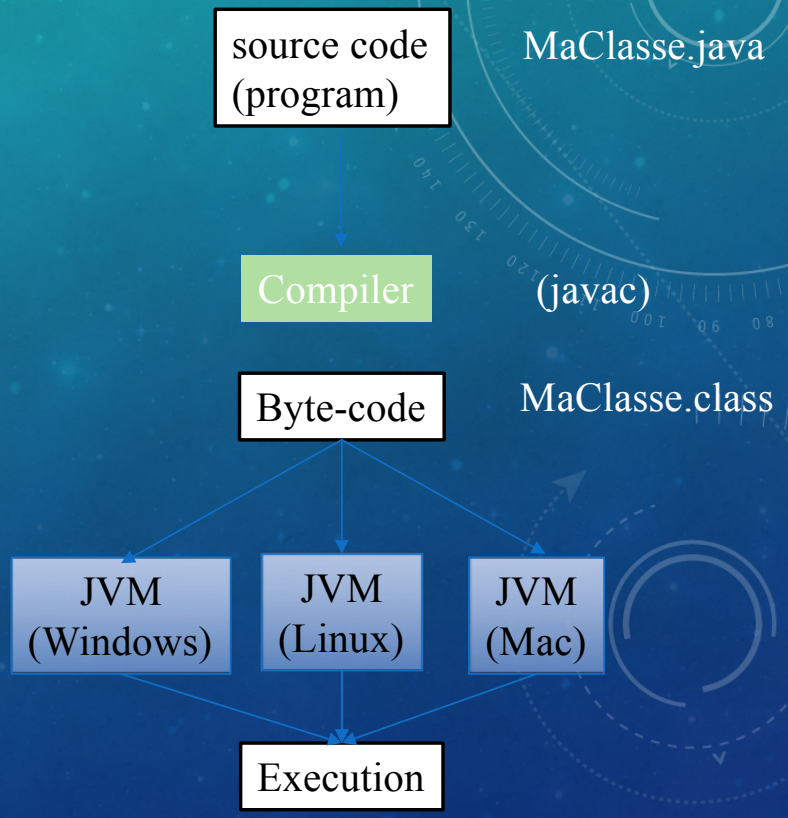



```
Car BMW = new Car ("X1123",  "white", 110);
```

Object of the class Car

JAVA LANGUAGE

- JAVA is a cross-platform compiled and interpreted language (independent of hardware and software architecture).
- The JAVA compiler, **Javac**, creates portable pseudocode: the **bytecode**.
- The bytecode is executed by a software program: the Virtual Machine **JVM**.
- The JVM is available on Linux, Mac, Windows, and other platforms (Portability).



WHY JAVA?

- Simplicity
- Security: essential on networks (protection against viruses, modification of files, reading of confidential data, etc.).
- Strong (robust)
- Distributed: Java supports remote communication mechanisms through its network API (Application Programming Interface Java).
- Multi-thread: A Thread is a stream of instructions executing concurrently with other threads in the same process.
- A rich bookstore.

JDK (JAVA DEVELOPMENT KIT)

- A set of tools and APIs to enable program development with Java:
  - JRE : Java Runtime Environment.
  - Standard class libraries: java.lang, java.util, etc.
  - The compiler (javac)
  - An interpreter (java) for running applications
  - An applet interpreter (Appletviewer)
  - A documentation generator (javadoc): /** add comment
  - A minimal debugger (jdb)

EXAMPLE OF DEVELOPMENT TOOLS

Development environment:

- Eclipse
- IntelliJ IDEA
- Apache Netbeans
- BlueJ, etc.

GENERAL STRUCTURE OF A JAVA APPLICATION: EXAMPLE "HELLO WORLD" PROGRAM

File Hello.java

The file name must be the class name, with the **.java** extension.
Java is case-sensitive.

```
public class Hello
{
//attributes declaration
//methods definition
public static void main (String args [])
    {
        //local variables declaration
       System.out.println("Hello I4.0");
    }
}
```

The class is the basic unit of our programs. The keyword to define a class in Java is **class**.

The method can receive parameters. Here, the **main method** receives the **args** parameter which is an array of strings.

COURSE OUTLINE

- Introduction
- Object Oriented Programming: basic concepts (syntax, types, variables, operators, etc.)
- The principles of Object Oriented Programming: Object, classes, access.
- Inheritance and Polymorphism
- Interfaces

CHAPTER 1: BASIC CONCEPTS

TYPE, VARIABLES, OPERATORS, ETC.

IDENTIFICATIONS AND RESERVED WORDS

Identifier Conventions:

- The first letter is lowercase for methods, attributes, and variables. Example: setLength, i, aMethod.
- If several words are joined together, each word except the first must be capitalized.

  Example: anIntegerVariable

IDENTIFICATIONS AND RESERVED WORDS

- The first letter is capitalized for classes and interfaces.

  Examples: MyClasse, TheGoodJob

- Constants are all uppercase

  Example: LENGHT_MAX

  A constant is declared with the "*final*" modifier. It is mandatory to initialize it at the time of declaration.

  Example: *final double PI=3.141592;*

- Reserved words

An identifier cannot belong to the list of words reserved for the Java language : *abstract*, *default*, *goto*, *null*, *synchronized*, *boolean*, *do*, *if*, *pachage*, etc.

TYPES

- Every value/expression has a type
- Example:
  - "hello": String type
  - 27: int type
  - 'x': char type
  - true: Boolean type

TYPES

Type conversion

- Legal implicit conversions in Java are:
  - byte → short → int → long → float → double
  - char → int → long → float → double
- Conversions that may cause a loss of value are not allowed implicitly. They must be requested explicitly (e.g., float to int).

© Soumaya Trabelsi

TYPES

Examples of conversions:

```
int i, j;
short s = 2;
float f =1.5;
i = (int) f; // mandatory explicit lossy conversion
i = s; // implicit conversion
f = i/2.0; // conversion by arithmetic promotion
// let be the method int meth(int arg)
j = objet.meth(s); //conversion when passing parameters
```

© Soumaya Trabelsi

VARIABLES (ATTRIBUTES)

- Stores a value
- Can be used in place of the value it stores
- Defining a variable:

  TypeName variableName = value; or *TypeName variableName;*

- For example:

```
String greeting = "Hello, Dave!";
```

EXERCISE 1

- What will the following program display?

```
public class EX1
{ public static void main( String args[])
   {
          int a,b,c;
          boolean resultat;
          a=3;
          b=3;
          c=4;
          resultat = (a>b);
          System.out.println(resultat);
          resultat = (a!=c);
          System.out.println(resultat);
          System.out.println( (a != c) );
          System.out.println( (a >= b) );
          System.out.println(!(a < c);
   }
}
```

```
false

true
true
true
false
```

EXERCISE 2

- What is the following program supposed to do? Correct it if necessary.

```
public class EX1
{ public static void main( String args[])
    {
        int a,b;
        double m;
        a=4;
        b=7;
        m=(a+b)/2;
        System.out.println(m);
    }
}
```

Solution: to get the correct result (5.5), at least one of the numbers of the division must be a double or a float:

m = (a+b)/2.0;

or

m= (double) (a+b)/2; *(Casting)*

Integer Division Issue:

- (a+b) and 2 are both integers
- (a+b)/2 is an integer
- Expected result: 5.5
- Real result: 5.0, even though ***m*** is a double.
- In Java, when you divide an integer by another integer, the result is always an integer. Any decimal part is truncated.

EXERCISE 3

- Write a program that simply swaps two integers a=5 and b=8

```java
public class SwapNumbers {
    public static void main(String[] args) {
        // Declaration and initialization
        int a = 5;
        int b = 8;
        int temp;

        System.out.println("Before swapping: a = " + a + ", b = " + b);

        // Swapping process using a temporary variable
        temp = a; // temp now holds 5
        a = b;    // a now holds 8
        b = temp; // b now holds 5

        System.out.println("After swapping: a = " + a + ", b = " + b);
    }
}
```